

Operating Memory IV: Build the Memory Layer

FRIDAY AI Education — Episode 030 · 2026-07-16 · Printable Companion

FRIDAY AI Education — Episode 030 · Printable Paper

Lesson 16 of 37 · Operating Memory, part 4 of 4

Abstract

The previous three papers in this unit established what organizational memory must do: preserve evidence, track who said what and with what authority, handle time honestly, and retrieve selectively rather than exhaustively. This paper does the remaining work: it assembles those principles into a single buildable design. The architecture has four layers — an evidence register that preserves originals, a claim ledger that records structured assertions with provenance and authority, a conflict-and-supersession layer that keeps disagreement visible, and a retrieval policy that assembles a small context packet for the model instead of dumping the corpus. We build one complete memory from three real-world inputs (a policy document, a meeting correction, and a stale summary), specify exactly which artifact lives in Git, in structured records, in the semantic index, and in the prompt, and trace one query from raw evidence to final model context. By the end, you should be able to sketch this system on one page and defend every design decision in it.

1. Where We Are

Three lessons of theory precede this one. Operating Memory I established why raw documents are not memory. Operating Memory II gave us authority, provenance, and conflict as first-class properties of any recorded claim. Operating Memory III covered retrieval, forgetting, and the learning loop — the idea that a memory system is judged by what it *surfaces*, not what it *stores*.

The single idea this paper extends is the one from the last lesson: **retrieval is a policy decision, not a search feature**. Today we build the thing that policy operates on.

This is the synthesis lesson. Almost nothing here is conceptually new; what is new is the *shape* — the specific set of artifacts, the boundaries between them, and the discipline of knowing which layer each piece of information belongs to. That sounds modest. It is not. Most organizational AI projects fail precisely here: the concepts are understood in the abstract and then everything gets thrown into one vector database anyway, because building the layers felt like overhead. The overhead is the product.

One framing note before we start. The design below is deliberately small. It fits in a Git repository, a handful of structured records, and a semantic index. Resist the urge to reach for a knowledge-graph platform or an enterprise memory product on day one. The layers matter; the tooling mostly does not.

2. The Design Problem, Stated Precisely

Here is the problem in operational terms. An organization produces information continuously: policy documents, meeting decisions, corrections, summaries, contracts, emails. Some of it is authoritative, some is commentary, some

is wrong, and some was right and no longer is. An AI system operating inside this organization needs to answer questions and take actions based on what the organization currently holds to be true — while remaining able to show its work when challenged.

That sentence contains four separable requirements:

1. **Preserve the originals.** You can never reconstruct evidence you summarized away. Summaries are derived artifacts; the source is the asset.
2. **Record claims, not just documents.** "What does the expense policy say?" is a question about a *claim* — a specific assertion with an author, a date, an authority level, and a validity window. Documents contain claims; they are not the claims themselves.
3. **Keep conflict visible.** When two sources disagree, the system must represent the disagreement rather than silently picking a winner at ingestion time. Silent resolution is where trust dies, because nobody can later see that a choice was made.
4. **Retrieve small and precise.** The model should receive a curated context packet — the winning claim, its provenance, any live conflicts, and nothing else — not forty loosely related chunks.

Each requirement maps to one layer of the architecture. The rest of this paper builds them in order.

3. Layer One: The Evidence Register

The evidence register is the foundation, and it is almost embarrassingly simple: a store of **original artifacts, unmodified, each with a stable identifier and capture metadata.**

An entry in the evidence register records:

- **Evidence ID** — a stable, permanent identifier (EV-2026-0142).
- **The artifact itself** — the PDF, the meeting transcript, the email, byte-for-byte as received.
- **Capture metadata** — when it was captured, from where, by whom or by what process.
- **A content hash** — so you can later prove the artifact has not been altered.

The register's one rule: **evidence is append-only.** You never edit an entry, never "clean up" a transcript, never replace a document with a newer version in place. A new version of a document is *new evidence* with its own ID. The old version stays, because claims made last quarter were made against last quarter's document, and you will eventually need to explain a decision that was correct at the time and looks wrong now.

This is the layer people skip, and the skip is always rationalized the same way: "the summary captures everything important." It does — until a dispute arises about what the source actually said, at which point the summary is worth exactly nothing as evidence. Lawyers understood this centuries before software engineers did. The register is your chain of custody.

Practically, for a small operation, the evidence register is a directory in Git: files plus a small metadata sidecar per file. Git gives you the append-only history, the hashes, and the timestamps nearly for free. There is no need for anything more exotic until the volume forces it.

4. Layer Two: The Claim Ledger

Documents are inert. To operate on knowledge, you extract **claims**: atomic, structured assertions of the form "X holds, according to source S, asserted by A, valid from T."

The claim ledger is the structured record of every claim the organization has registered. Each claim carries:

- **Claim ID** — stable identifier (CL-0871).
- **Statement** — the assertion, in one plain sentence. Atomic: one fact per claim. If your statement contains "and," it is probably two claims.
- **Provenance chain** — the path from this claim back to raw evidence: which evidence entry it derives from, who or what extracted it, and any intermediate transformations. If a claim came from a summary that came from a transcript, the chain records both hops. A claim with a broken provenance chain is a rumor with a database row.
- **Authority** — the level of the asserting source under your authority policy (next section).
- **Valid time** — when the claim is true *in the world*, as distinct from when it was recorded. A policy approved on March 3 and effective April 1 has a recorded time of March 3 and a valid time starting April 1. Conflating these two clocks is one of the most common and most damaging errors in memory system design. Valid time also has an end: a claim can expire or be closed by supersession.
- **Status** — active, superseded, disputed, or expired.

Note what the ledger is *not*: it is not a mirror of every sentence in every document. You register claims that matter operationally — policies, decisions, commitments, key facts. The extraction step is where human or AI judgment enters, and it is exactly the step where an LLM assists well: proposing claim extractions from evidence for a human to approve. Proposing, not committing. The ledger is a governed surface.

4.1 The Authority Policy

Provenance tells you *where* a claim came from. Authority tells you *how much that source is allowed to settle the matter*. These are different questions, and collapsing them is a category error: a perfectly provenanced claim from an intern's Slack message is still an intern's Slack message.

An **authority policy** is an explicit, written ranking of source types for each domain of claims. For example:

1. Signed policy documents and board resolutions
2. Decisions recorded in official meeting minutes
3. Statements by the accountable executive in writing
4. Team documentation and wikis
5. Informal messages and individual recollection

Two properties make it a *policy* rather than a *vibe*. First, it is written down before conflicts occur, so resolution is rule-driven rather than negotiated per incident. Second, it is domain-scoped: the CFO is the top authority on expense thresholds and roughly nobody on deployment procedure. A single global ranking is a design smell.

The authority policy is a document — versioned prose in Git, owned by a human. The system reads it; it does not write it.

5. Layer Three: Conflict Sets and Supersession

Now the part most systems get wrong. Two claims about the same subject will eventually disagree. The design question is *where the disagreement is resolved and whether the resolution is visible*.

The wrong answer, near-universal in practice, is to resolve at ingestion: the pipeline sees a newer statement, overwrites the older one, and the disagreement evaporates. This feels like tidiness. It is actually evidence destruction — you have lost the fact that the organization ever disagreed with itself, which is often the most operationally interesting fact available.

The right answer has two mechanisms, and it is worth being precise about which applies when.

5.1 The Conflict Set

When two active claims about the same subject are incompatible and *neither clearly replaces the other*, they are grouped into a **conflict set**: a named record that says "these claims disagree; here is the axis of disagreement; here is the resolution status." Both claims stay in the ledger with status disputed. The conflict set is a first-class object — it can be retrieved, listed, assigned to a human, and closed.

This is the mechanism for genuine uncertainty: the sales deck says one number, the finance model says another, and no rule cleanly decides. The system's job is to *hold the tension honestly* and surface it, not to flip a coin with confidence.

5.2 Supersession

When a newer claim *legitimately replaces* an older one — a policy update, an explicit correction by an authorized person — the mechanism is **supersession**: a directed link, CL-0912 *supersedes* CL-0871, plus a status change on the old claim and a closing of its valid-time window. The old claim is not deleted. It remains queryable ("what was the policy in Q1?") and remains part of the provenance story of every decision made under it.

The discipline: **supersession requires authority**. A newer statement from a lower-authority source does not supersede; it opens a conflict set. A newer statement from a source with sufficient authority under the policy supersedes cleanly. The authority policy is what turns "newer" from a heuristic into a rule. Recency alone settles nothing — the freshest statement in most companies is also frequently the least considered one.

So the full decision procedure when a new claim arrives about an existing subject:

1. Is it compatible with active claims? → Register alongside them. Done.
2. Incompatible, and the new source has authority to replace? → Supersede: link, close the old claim's valid time, mark it superseded.
3. Incompatible, without clear authority to replace? → Open a conflict set, mark both disputed, queue for human resolution.

Three branches. A junior engineer can implement this in an afternoon. The value is not in the code; it is in the fact that the third branch *exists*.

6. Layer Four: Retrieval Policy and the Context Packet

Storage is now solved in principle. The remaining question is what the model actually sees.

The previous lesson established that retrieval is governed selection, not similarity search. Here is what that means concretely: a **retrieval policy** is a written specification of, for a given class of query, *what gets included in context, in what order of precedence, and what gets excluded* — semantic similarity being one input among several, alongside authority, valid time, claim status, and conflict membership.

The output of the retrieval policy is a **context packet**: a small, structured bundle assembled per query. A well-formed context packet contains:

1. **The governing claims** — active, valid-now, highest-authority claims answering the query. Usually a handful, not a corpus.
2. **Their provenance** — source identifiers and dates, so the model can cite and the human can verify.
3. **Live conflicts, flagged** — if a relevant conflict set is open, the packet says so explicitly and includes both sides. The model is instructed to surface the dispute, not adjudicate it.
4. **Supersession notes where relevant** — "this replaced an earlier policy on date X" when the change itself is likely material to the question.
5. **Explicit boundaries** — what the packet does not cover, so the model can say "outside my provided context" instead of improvising.

The contrast with corpus-dumping is worth dwelling on, because dumping is the default behavior of every naive retrieval pipeline. Stuffing forty chunks into context feels thorough and is actually an abdication: it silently delegates authority resolution, temporal reasoning, and conflict handling to the language model at inference time — the one component in your stack with no persistent memory, no access to your authority policy, and a documented fondness for the most fluent-sounding passage. Every judgment your memory layer fails to make before the prompt is a judgment the model makes for you, invisibly, differently each time.

The context packet is where all four layers cash out. Everything upstream exists so that this small bundle can be *right*.

7. Worked Example: One Company-Policy Memory

Enough architecture. Let's build one memory from realistic inputs.

The scenario. Your company has a travel-and-expense policy. Three artifacts exist:

- **Artifact A** — *Expense Policy v2* (PDF, signed by the CFO, dated 2026-01-15): "Any single expense above €1,000 requires CFO approval prior to commitment."
- **Artifact B** — *Leadership meeting minutes* (2026-05-10): "Decision: to unblock vendor onboarding, the CFO-approval threshold for the engineering department is raised to €2,500, effective June 1. CFO present and approving. Policy document to be updated." (It was not updated. It never is.)
- **Artifact C** — an *onboarding wiki page* (last edited 2025-08-02): "Expenses over €500 need CFO sign-off." A stale summary of Policy v1, which no longer exists as an active document.

This trio — authoritative document, verbal correction with authority, stale derived summary — is the canonical hard case. Most companies' real answer to "what's the expense threshold?" lives in exactly this kind of mess.

Step 1 — Register the evidence. All three artifacts enter the evidence register unmodified: EV-101 (the PDF), EV-102 (the minutes), EV-103 (a capture of the wiki page). Yes, the stale wiki page too. It is evidence of what the

organization has been telling new employees, which is operationally important even though — especially because — it is wrong.

Step 2 — Extract claims.

- CL - 201, from EV - 101: "Single expenses above €1,000 require prior CFO approval." Authority: signed policy (level 1). Valid from 2026-01-15.
- CL - 202, from EV - 102: "For the engineering department, the CFO-approval threshold is €2,500." Authority: minuted decision with the accountable executive present (level 2, with explicit CFO assent noted in the provenance chain). Valid from 2026-06-01 — note the valid time is the effective date, not the meeting date.
- CL - 203, from EV - 103: "Expenses above €500 require CFO sign-off." Authority: wiki (level 4). The provenance chain notes it as a derived summary with its underlying source no longer active.

Step 3 — Run the conflict procedure.

- CL - 203 vs CL - 201: incompatible. Does the newer, higher-authority CL - 201 supersede? Yes — a signed policy outranks a wiki summary, and it is later. CL - 203 is marked **superseded**, valid time closed at 2026-01-15. It remains in the ledger. (Separately, the system can now flag that a live wiki page contradicts active policy — a small operational gift that corpus-dump systems structurally cannot give you, because they never *know* anything is stale.)
- CL - 202 vs CL - 201: incompatible *for engineering*. Does a minuted decision supersede a signed policy? Here your authority policy earns its keep. If it says "minuted decisions with the accountable executive's recorded assent may scope-modify signed policy," then CL - 202 **partially supersedes** CL - 201: the ledger records that CL - 201 remains active generally, with an engineering-scoped exception from June 1. If your authority policy says only a signed document amends a signed document, you instead open a **conflict set** — "engineering threshold: €1,000 (policy) vs €2,500 (minuted decision)" — and route it to a human, probably with the suggestion that someone finally update the PDF.

Either outcome is defensible. What is not defensible is the outcome you get without this machinery: the answer depends on which chunk the similarity search happened to rank first that day.

Step 4 — Answer a query. Query: "*Does a €1,800 software purchase by an engineer need CFO approval?*" The retrieval policy filters to active claims about expense approval, valid as of today, ranks by authority, checks conflict membership, and assembles a context packet: CL - 201 (general rule), CL - 202 (engineering exception, effective June 1), both with sources and dates, plus the supersession note on the old €500 figure in case the user has seen the wiki. The model answers: no CFO approval needed — €1,800 is under the €2,500 engineering threshold in effect since June 1, per the leadership decision of May 10; the general threshold remains €1,000; the €500 figure on the wiki is obsolete. Cited, dated, checkable.

That answer is not the product of a smarter model. It is the product of a memory layer that had already done the thinking.

8. Where Everything Lives

A recurring practical confusion: which artifact belongs in which substrate? The answer follows from what each substrate is good at. Git is good at immutable history and human-readable review. Structured records are good at fields you filter and join on. A semantic index is good at "find candidates by meaning." The prompt is good at exactly one thing: being read by the model right now.

Artifact	Git / Markdown	Structured claim records	Semantic index (GBrain)	The prompt
Original evidence (PDFs, transcripts)	✓ stored, append-only	ID + hash reference only	indexed for candidate search	never — only cited excerpts
Evidence metadata (capture info, hashes)	✓ sidecar files	✓	—	—
Claims (statement, status, valid time)	optional human-readable mirror	✓ system of record	statement text, for discovery	✓ the governing few
Provenance chains	—	✓ links to evidence IDs	—	✓ compressed: source + date
Authority policy	✓ versioned prose, human-owned	encoded ranks derived from it	—	only its <i>verdicts</i>
Conflict sets	—	✓	—	✓ if live and relevant
Supersession links	—	✓	—	✓ as a note, when material
Retrieval policy	✓ versioned prose + config	—	—	never (it <i>builds</i> the prompt)
Context packet	logged per query, for audit	—	—	✓ this <i>is</i> the prompt payload

Three observations worth pinning. First, the semantic index is deliberately demoted: it is a *candidate finder*, one stage inside the retrieval policy, not the memory itself. Embeddings represent similarity, not truth, and nothing in this design asks them to do more. Second, the prompt row is nearly empty — by design. The prompt is a rendering surface, not a storage layer; anything that must be durable lives upstream. Third, everything a human must be able to read, review, and version — evidence, authority policy, retrieval policy — lives in Git as plain text. Governance surfaces belong where diffs and reviews happen.

9. One Query, Traced End to End

To make the pipeline concrete, here is the full path of the query from Section 7, stage by stage:

1. **Query intake.** "Does a €1,800 software purchase by an engineer need CFO approval?" Classified as a policy question in the expense domain — which selects the retrieval policy variant to apply.
2. **Candidate search.** The semantic index returns everything meaning-adjacent: the three claims, plus noise — a Slack thread about laptop budgets, last year's travel policy discussion. This stage is intentionally generous; precision comes later.
3. **Status and time filter.** Superseded claims drop (CL - 203 exits here — this single filter is where the stale wiki answer dies). Claims not valid *today* drop. The noise mostly lacks claim status at all and drops as unregistered.
4. **Authority ranking.** Survivors are ordered by the authority policy: signed policy first, minuted decision second.
5. **Conflict check.** Are any survivors members of an open conflict set? If yes, both sides enter the packet with an explicit dispute flag and the model will be instructed to present, not resolve. In our resolved version of the example: no open conflicts; the engineering exception carries its supersession context instead.
6. **Packet assembly.** Governing claims, provenance summaries, the supersession note, and scope boundaries are rendered into a compact structured block — typically well under a page.

7. **Generation.** The model receives the packet plus instructions: answer from the packet, cite claim sources and dates, surface any flagged conflicts, and decline to speculate beyond the packet's stated boundaries.
8. **Audit trail.** The packet itself is logged with the query and response. When someone later asks "why did the system say no approval was needed?", the answer is inspectable: here is exactly what the model was shown, and here is the evidence behind each claim in it.

Notice the division of labor. Stages 1–6 are deterministic, testable, boring software — you can write unit tests asserting that a stale claim never survives stage 3. Stage 7 is the only place a language model reasons, and by the time it does, the hard epistemic work — what is true, what is authoritative, what is current, what is disputed — is already done. The model is drafting the memo, not deciding the facts. That is precisely the posture you want, and it is also why this lesson leads directly into the next unit topic: a system that acts on memory is only as safe as the memory discipline underneath it. Unsafe action usually begins as unexamined context.

10. Failure Modes and Honest Limitations

No design survives contact with reality unbruised. The ways this one degrades, roughly in order of likelihood:

Extraction is the bottleneck and the weak joint. Claims do not extract themselves. If registration requires heavy ceremony, people stop doing it, and the ledger silently diverges from reality — which is worse than having no ledger, because it *looks* authoritative. Mitigations: keep claim records small, let an LLM propose extractions from new evidence with human approval, and scope ruthlessly. You do not need every fact registered. You need the fifty claims your operation actually turns on.

The authority policy meets politics. Writing down that meeting minutes can override a wiki is easy. Writing down whose statement overrides the COO's is a governance question wearing a schema costume. Expect the first real conflict-set resolutions to be organizationally interesting. This is a feature — the system is surfacing ambiguity that was always there — but budget for the conversations.

Scope creep in claim granularity. Teams either register claims too coarse to resolve conflicts ("the expense policy exists") or so fine that maintenance collapses. The test: a claim should be exactly the thing a conflict could be *about*. If two sources could never disagree over it, it may not need to be a claim.

Conflict sets that never close. An open conflict set is honest for a week and negligent for a quarter. Unresolved sets need an owner and an aging alarm, or "represented, not resolved" quietly becomes "ignored, with paperwork."

The system answers confidently outside its ledger. The packet's stated boundaries help, but a model will still generalize. Evaluations — regression queries with known-correct, known-cited answers, run on every change to the retrieval policy — are the standing defense. Memory infrastructure without evaluations is a claim about correctness with no provenance chain, which by this point in the unit should make you flinch.

None of these are reasons to skip the design. They are reasons to start with one narrow domain — one policy area, fifty claims, one retrieval policy — and let the machinery earn its way into broader scope. The architecture scales by addition, not redesign.

11. Vocabulary

- **Evidence register** — Append-only store of original artifacts, each with a stable ID, capture metadata, and content hash. The layer nothing is allowed to edit.

- **Claim ledger** — The structured record of atomic assertions extracted from evidence, each carrying provenance, authority, valid time, and status. The system of record for what the organization holds true.
- **Provenance chain** — The traceable path from a claim back through any derivations to its source evidence. A claim without one is a rumor with formatting.
- **Authority policy** — A written, domain-scoped ranking of source types that determines which claims can settle or supersede others. Owned by humans, read by the system.
- **Valid time** — The window during which a claim is true in the world, as distinct from when it was recorded. The two clocks of any honest memory system.
- **Conflict set** — A first-class record grouping incompatible active claims when no rule cleanly resolves them; makes disagreement visible, ownable, and closable.
- **Supersession** — An authorized, directed replacement of one claim by another, closing the old claim's valid time while preserving it for history. Requires authority, not just recency.
- **Retrieval policy** — The written specification of what enters model context for a class of query — filtering by status, valid time, authority, and conflict membership, with similarity as one input among several.
- **Context packet** — The small, curated bundle a query actually produces: governing claims, provenance, flagged conflicts, and explicit boundaries. The entire architecture's output surface.

12. Reading Guide

W3C PROV Overview — <https://www.w3.org/TR/prov-overview/>

The web standard for representing provenance. Read the overview page only — the family of full specs is for implementers of interchange formats, not for you. What to take: the core triad of *entities* (things, like our evidence and claims), *activities* (processes, like extraction), and *agents* (people and software responsible for them). Notice that our provenance chain is a lightweight instance of exactly this model — claims are entities *derived from* evidence entities *by* extraction activities *attributed to* agents. You do not need to adopt PROV's serializations; you need to recognize that provenance modeling is a solved design problem with settled vocabulary, so you can borrow its structure and skip its ceremony. Fifteen minutes, once.

NIST AI Risk Management Framework —

<https://www.nist.gov/itl/ai-risk-management-framework>

Read for framing, not implementation detail. The framework's core functions — govern, map, measure, manage — describe at organizational scale what this paper builds at data scale. The connection to draw: nearly everything the RMF asks of a trustworthy AI system (traceability, documented data lineage, transparency about limitations, human oversight at consequential points) is *structurally provided* by the layers in this paper. The evidence register and provenance chains are traceability; the authority policy and conflict sets are governance made mechanical; logged context packets are transparency. Skim the framework's function descriptions and map each to a layer here. This is also the vocabulary that makes you credible in rooms where "we have good retrieval" is not an acceptable answer to a risk question.

13. Walking Questions

Take these on the next walk — no notes, just reasoning:

1. Explain, without looking back, why supersession requires authority and not merely recency. What breaks in a company where the newest statement always wins?

2. Which layer could your current setup skip for six months with least damage — and which skip would be quietly catastrophic? Defend the asymmetry.
3. Pick one real policy or decision in your own operation that currently lives in the "policy doc plus verbal correction plus stale summary" state. What would its three claims and their authority levels actually be?
4. Where does this architecture show up in GBrain's role as substrate versus the orchestrator's role as operator — which layers belong to each?
5. What is one bounded workflow — small enough to be safe, real enough to matter — where you could run fifty claims through this design and evaluate whether the answers improve?

Takeaway

A memory layer is not a database of documents but a discipline of layers — preserved evidence, authored claims, visible conflict, governed retrieval — so that by the time the model reads its small context packet, every hard question about truth, authority, and time has already been answered by something accountable.